パッケージのメタデータと動的ロード由来の共有ライブラリの依存関係の差分評価

奥川 莞多[1,2,a)]　竹久 達也[1,3,2]　笠間 貴弘[1]　毛利 公一[2,1]

概要：オープンソースソフトウェアの利用拡大に伴い，ソフトウェアを構成する要素とその依存関係を正確に把握することの重要性が高まっている．Linux ディストリビューションでは，OS パッケージの依存関係は主にパッケージメタデータを通じて管理され，それをもとにインストール管理や脆弱性影響範囲の推定に利用されている．しかし，プログラムは dlopen などを用いて実行時に共有ライブラリを動的にロードする場合があり，特に動的ロードによる依存関係は静的に検出することが容易ではない．そのため，パッケージメタデータに基づく依存関係把握には，実行時に利用され得る構成要素を十分に表現できない可能性がある．本研究では，Debian パッケージを対象として，動的ロードによって生じる共有ライブラリ依存候補が，メタデータ上でどのように扱われているかを実証的に分析する．これにより，メタデータ等の静的な情報をパッケージの依存関係としてソフトウェア構成管理に用いる際の適用範囲と限界を明らかにする．

キーワード：動的ロード，ソフトウェア構成管理，パッケージ依存関係

Evaluating the Dependency Gap Between Package Metadata and Dynamically Loaded Shared Libraries

Kanta OKUGAWA[1,2,a)]　Tatsuya Takehisa[1,3,2]　Takahiro KASAMA[1]　Koichi MOURI[2,1]

Abstract: With the growing use of open-source software, it has become increasingly important to accurately understand the components that constitute software and the dependencies among them. In Linux distributions, dependencies between OS packages are primarily managed through package metadata, which is used for purposes such as installation management and estimating the potential impact of vulnerabilities. However, programs may dynamically load shared libraries at runtime using mechanisms such as dlopen, and dependencies introduced through such dynamic loading are particularly difficult to detect statically. Consequently, dependency information based solely on package metadata may not fully capture components that can potentially be used at runtime. In this paper, we empirically analyze Debian packages to investigate how potential shared-library dependencies arising from dynamic loading are represented in package metadata. Through this analysis, we clarify the applicability and limitations of using package dependency metadata for software component management.

Keywords: Dynamic Loading, Software Component Management, Package Dependencies

1. はじめに

オープンソースソフトウェア（OSS）の利用拡大に伴い，ソフトウェアは多数のライブラリや外部コンポーネントに依存して構成されている．Linux ディストリビューションにおけるパッケージは，実行ファイルや共有ライブラリ，設定ファイルなど複数のコンポーネントから構成されている．そのため，ソフトウェアサプライチェーンリスクや脆弱性管理において，ソフトウェアを構成するコンポーネントとそれらの依存関係を正確に把握することが重要であ

[1] 国立研究開発法人情報通信研究機構, National Institute of Information and Communications Technology
[2] 立命館大学, Ritsumeikan University
[3] 株式会社ニッシン, Nissin Inc.
[a)] kokugawa@nict.go.jp

© 2026 Information Processing Society of Japan

This paper is work in progress and not peer-reviewed.

る．しかし，ソフトウェアを実行する際に利用される構成要素は，必ずしも静的な情報のみから一意に決定できるとは限らない．実行時の設定や環境変数，プラグイン機構などによって，実際にロードされるコンポーネントが変化する場合がある．したがって，実行ファイルなどから取得される静的な構成情報と，実行時に利用され得るソフトウェア構成との間には差異が生じるという問題が発生する．

Linux ディストリビューションでは，パッケージ間の依存関係は主にパッケージメタデータによって管理される．例えば，Debian では，パッケージの Depends や Pre-Depends などのフィールドに実行時に必要となる依存パッケージが記述され，パッケージマネージャーはこれらの情報を用いて必要なパッケージのインストールやアップデートを行う．このようなパッケージメタデータやバイナリに含まれる依存情報は，ソフトウェアの構成要素を把握する上で有用な情報であり，SBOM を用いた脆弱性影響範囲の特定などにも利用される．

一方で，Linux 上のプログラムは dlopen(3) を用いることで，プログラムの実行時に任意の共有ライブラリを動的にロードできる [1]．動的ロードは，必要な機能が使用された場合のみに共有ライブラリをロードすることができるため，プラグイン機構の実装やオプション機能の提供に利用されている．機能のモジュール化や資源利用の削減を目的に利用されるケースが多いが，不要な共有ライブラリをプロセスへ常時ロードすることによるリスクからソフトウェアサプライチェーンリスクの観点でも注目されている [2]．

しかし，dlopen による動的ロードには，依存関係の把握を困難にするという問題が存在する．動的リンクによる共有ライブラリ依存は ELF セクションに記録されるのに対し，dlopen によってロードされるライブラリは，静的な情報だけからは特定できず，実行時まで確定しない．さらに，ロード対象がオプション機能やプラグインとして扱われる場合，対応するパッケージを Depends や Recommends などのどの依存関係として記述するか，記述しないかは，開発者やパッケージ作成者による明示的な記述に依存する．このため，実行時に利用され得る共有ライブラリが，ELF の動的リンク情報やパッケージメタデータから把握できない可能性がある．この問題に対して，UAPI Group は dlopen 対象の SONAME などを ELF の.note.dlopen へ記録する仕様を定めている [3]．Debian 13 では，この情報を解析する dh_dlopenlibdeps が提供されており，パッケージが利用する依存情報を生成できる [4]．これにより，動的ロード依存を機械的に判別できるが，.note.dlopen の仕様上，開発者が明示的にプログラム上に記述する必要があり，依然として記載される使用ライブラリの精度は開発者に委ねられる．

そこで，本研究では，実行時に動的に決定されるソフトウェア構成の形態として，共有ライブラリの動的ロードに着目し，静的に表現されるソフトウェア構成と実行時に利用され得るソフトウェア構成との差異を明らかにする．具体的には，大規模な Debian のパッケージを対象として，ソースコードの中間表現を解析することで dlopen の利用を抽出し，動的ロードの対象となるライブラリ名を可能な範囲で静的に特定する．さらに，取得したライブラリを提供するパッケージを特定し，呼び出し元パッケージのメタデータに記載される依存関係と比較する．加えて，ELF の .note.dlopen による宣言状況についても調査する．これらの分析を通して，Debian パッケージエコシステムにおいて動的ロードがどの程度利用され，その依存関係が静的な情報によってどの程度把握可能かを明らかにすることを目的とする．結果として，動的ロードによる依存関係の一部は静的解析によって特定可能である一方，既存のパッケージメタデータのみでは把握できない依存関係が十数パッケージ存在することを確認した．本研究の主な貢献は以下の通りである．

- **Debian パッケージにおける動的ロード利用の大規模な実態把握：**Debian 12/13 のソースパッケージにおける `dlopen()` の利用を抽出し，動的ロードを利用するパッケージの定量化や傾向分析を行う．
- **動的ロード由来の共有ライブラリの静的分析：**dlopen 等の引数を静的に解析し，使用ライブラリ名を可能な範囲で特定することで，静的に取得可能な依存関係と，実行時に決まる依存関係の境界を明らかにする．
- **動的ロード依存とパッケージメタデータとの差分分析：**抽出したライブラリを提供するパッケージを特定し，メタデータのパッケージ依存関係と比較することで，既存の依存関係管理によって把握できる範囲と追加確認を必要とする範囲を明らかにする．

2. 背景と関連研究

2.1 動的ロードによる共有ライブラリの利用形態

ライブラリの利用形態には，表 1 に示すように静的リンクと通常の動的リンクに加えて，dlopen 等を用いた動的ロードがある．動的リンクでは，リンク対象となる共有ライブラリの SONAME が ELF の DT_NEEDED に記録され，プロセス起動時に動的リンカによってロードされる．遅延リンクを利用する場合にはシンボル解決が初回呼出し時まで遅延されるが，対象となる共有ライブラリ自体は ELF に記載されるため，動的リンクによる直接的な共有ライブラリ依存は ELF から静的に取得できる．

一方，dlopen による動的ロードでは，プログラムの実行中にロード対象の共有ライブラリが指定される．その引数は固定文字列として記述される場合だけでなく，設定ファイルや実行時の処理結果から決定される場合もある．このような共有ライブラリは通常 DT_NEEDED には記録されない上，実行時まで使用するライブラリが確定しない

© 2026 Information Processing Society of Japan

表 1: ライブラリ利用方式と静的な観測点

方式	使用タイミング	主な観測点
静的リンク	リンク時	実行ファイル内
動的リンク	プロセス起動時（遅延解決も可能）	DT_NEEDED
動的ロード	dlopen 実行時	引数や.note.dlopen

可能性があるため，ELF に記載される静的な情報のみからロード対象の共有ライブラリを把握することは容易ではない [5]．例えば，Apache では，動的共有オブジェクト（DSO）として提供されるモジュールを設定に応じてロードすることで，必要な機能を選択してサーバを構成できる．

2.2 Debian における共有ライブラリの依存関係管理

Debian では，1 つのソースパッケージから 1 つ以上のバイナリパッケージが生成される．ソースパッケージの Build-Depends はビルド時に必要なパッケージを表すのに対し，実行時の依存関係は各バイナリパッケージの Depends や Pre-Depends などで管理される．そのため，同一のソースパッケージから生成されたバイナリパッケージであっても，収録する実行ファイルやライブラリ，プラグインによって実行時の依存関係は異なる．また，通常の動的リンクによる共有ライブラリ依存について，Debian では ELF の情報をもとに依存パッケージを生成する仕組みが用意されている．パッケージのビルド時に dh_shlibdeps が呼び出され，ELF ファイルを解析対象に使用している共有ライブラリを取得し，${shlibs:Depends} として対応するパッケージ名と必要なバージョンが依存パッケージとして適用される．この仕組みは DT_NEEDED に記録された共有ライブラリ依存を対象としているため，dlopen によって実行時に指定される共有ライブラリは解析対象とはならない．そのため，動的ロードによる依存関係は，開発者がパッケージの依存関係として明示しない限り，通常の共有ライブラリ依存の解析だけでは把握できない場合がある．

この問題に対応するため，近年では dlopen によって利用する共有ライブラリを ELF 上に宣言する.note.dlopen が導入され [3]，Debian でもその情報から依存関係を生成する dh_dlopenlibdeps が提供されている [4]．また，package-notes[6] は deb と rpm 向けのパッケージにおいて，ELF に対するパッケージ情報の生成と.note.dlopen の抽出をサポートしている．しかし，.note.dlopen は実際の dlopen 呼出しから自動的に導出されるものではなく，ソフトウェア側が動的ロード関係を明示するための情報である．したがって，実際に存在する動的ロード依存が.note.dlopen にどの程度反映されているかは開発者に依存しており，その実態は明らかではない．

2.3 関連研究

ソフトウェアの構成要素を正確に把握するため，静的なメタデータだけでなく，バイナリ解析や実行時情報を用いて構成情報を取得するような研究が進んでいる．JBomAudit[7] は Java における SBOM と JAR ファイルの分析結果を比較し，依存関係の欠落や不備を分析している．また，LiPSBOMaker[8] は Linux パッケージについて複数段階から SBOM を生成する手法を提案している．MEM-SBOM[9] は Python プロセスのメモリからロード済みモジュールを復元し，実際に実行環境へロードされた構成要素を SBOM として生成する手法を提案している．これらの研究は静的情報のみでは，実行時のソフトウェア構成を十分に把握できない場合があることを示している．さらに，実行中のワークロードで利用されたパッケージやライブラリを静的 SBOM と対応付ける Real-time SBOM が提供されている [10], [11].

RTrace[5] は，共有ライブラリの実行を観測する手法として，ltrace や drltrace などの既存手法を比較し，不完全なシンボル情報，ロード中の早期呼出し，間接呼出しなどによって共有ライブラリの利用を見逃す場合があることを示している．これらの手法は，特定のワークロードにおいて実際に利用された構成要素を把握できる一方，実行されなかった機能やプラグインが潜在的にロードし得る共有ライブラリを網羅するものではない．

また，パッケージの依存関係や動的ロードについて調査した研究が存在する．DepEx[12] は，Ubuntu の複数のリリースを対象に，バイナリと共有ライブラリの関係およびパッケージ依存関係を分析している．Nibbler[13] は，共有ライブラリの Debloating を目的としてバイナリ解析手法を提案しており，その課題の 1 つとして dlopen や dlsym 等の動的ロードを挙げており，Debian 9 の 1,351 件程度のパッケージ内のソースコードに動的ロードが含まれていることを報告している．Nibbler は共有ライブラリの解析を目的としているのに対して，本研究では，動的ロードによって利用され得る共有ライブラリとパッケージメタデータ上の依存関係を対応付けることで，静的な情報から把握できない動的ロード由来の依存関係の存在を明らかにすることを目的としている．

3. 課題設定と Research Question

前章で述べたように，dlopen を用いた動的ロードは，プラグイン機構やオプション機能の実現に利用されており，不要なライブラリのロードを避けることで，資源利用やアタックサーフェス最小化の手段としても利用される可能性がある [2]．一方で，動的ロードによる共有ライブラリ依存は，通常の動的リンクとは異なり ELF の依存情報から自動的に取得できず，パッケージメタデータへの反映も明示的な宣言に依存する．そのため，実際に dlopen によって利

© 2026 Information Processing Society of Japan

用される共有ライブラリと，パッケージ管理システムが保持する依存関係との間に差異が生じる可能性がある．このような依存関係の欠落は，必要なライブラリがインストールされず，対象機能の利用時にロードに失敗する可能性があるだけでなく，パッケージメタデータ等から生成される SBOM から動的に利用される構成要素が欠落し，実際には利用され得る脆弱なコンポーネントを見落とす可能性がある．ここで，本研究では，動的ロードの利用実態，ロード対象の特定，依存関係の比較までを分析する上で，以下の Research Question を設定する．

RQ1．OS パッケージにおいて動的ロードはどの程度，どのように利用されているか？動的ロードの利用規模と分布を明らかにすることで，一部の特殊な実装に限られるのか，もしくは OS パッケージの依存関係を把握する上で無視できない規模で利用されているのかを明らかにする．

RQ2．動的ロードされる共有ライブラリ名は，どの程度静的に特定できるか？ロード対象の静的な特定可能性を明らかにすることで，動的ロード由来の依存関係を静的解析によって把握できる範囲と，より高度な解析や実行時情報による補完を必要とする範囲を明らかにする．

RQ3．動的ロード由来の共有ライブラリ依存は，パッケージメタデータ上の依存関係にどの程度反映されているか？動的ロードによって利用され得る依存とパッケージメタデータ等の静的解析により構築される依存関係情報との差異を明らかにすることで，静的なソフトウェア構成と依存関係把握の限界を明らかにする．

4. 実験方法

本章では，Debian パッケージにおける dlopen による共有ライブラリの依存関係をメタデータの依存情報と比較するための分析手法について説明する．

4.1 概要

調査手法の概要を図 1 に示す．本手法は，(1) パッケージ集合の作成とバイナリによる事前抽出，(2) dlopen 呼び出し抽出，(3) 共有ライブラリとメタデータにおける依存関係との比較，の 3 段階から構成される．なお，本手法は実行時にロードされ得るライブラリを網羅的に列挙するものではない．対象リリースのデフォルトビルドから静的に復元可能な動的ロード依存を抽出し，既存の依存情報との対応を調べるための大規模実験としてパイプラインを設計しており，自動的に検証できる範囲で実験を行っている．以降，これらの実験方法について詳しく説明する．

4.2 実験データセットの構築

まず，対象リリースに含まれるソースパッケージおよびバイナリパッケージの一覧を取得し，各パッケージをダウンロードして展開する．ソースパッケージと，そこから生成されるバイナリパッケージの対応関係を記録する．次に，各バイナリパッケージに含まれる実行ファイルおよび共有ライブラリなどの ELF ファイルを検出する．各 ELF について，シンボル情報から dlopen を直接参照しているかを確認し，該当する ELF を含むパッケージを後続解析の候補パッケージとして抽出する．本研究では ELF から直接確認できる参照を対象とし，関数ポインタや独自のラッパーを介して動的に解決される呼び出しはこの段階では対象外とする．また，各 ELF について.note.dlopen の有無とその内容を取得する．この情報は，既存の動的ロード依存管理機構の利用状況を確認するとともに，後段で実際に検出した dlopen 由来のライブラリ候補との比較に用いる．

表 2: dlopen の引数検出の分類

分類	判定内容
一意	引数の文字列が具体的に 1 つに絞られる
有限個	引数の文字列が具体的に有限個に絞られる
パターン	変数や文字列操作などでパターンとして認識できるが，確定はできない
未確定	有用な制約を取得できない
対象外	上記のいずれにも該当しない検出内容

4.3 動的ロードの検出

事前解析によって抽出したソースパッケージに中間表現を出力する GCC フラグを追加した状態で再ビルドし，GCC が生成する GIMPLE SSA を取得する．まず，すべての GIMPLE を走査し，dlopen の直接呼び出し箇所を抽出する．GIMPLE はプリプロセス後の表現であるため，定数マクロや対象ビルドで有効となる条件などは展開された状態で解析できる．一方，対象ビルドで無効な条件分岐や実際にコンパイルされないコードは解析対象に含まれない．本研究ではデフォルト設定でビルドを行っており，コンパイルに含まれない動的ロード機能は分析対象外とする．次に，中間表現上で dlopen の第 1 引数を解析し，引数が変数の場合はロード対象となるライブラリ名を追跡する．文字列リテラルは具体値として取得し，変数への代入では定義元を参照する．分岐によって複数の値が存在する場合には候補を併合する．解析結果は，ロード対象名の特定可能性に基づいて表 2 の 5 種類に分類する．

4.4 メタデータとの比較

前節で検出した動的ロードのうち，最終的なバイナリパッケージに含まれることを確認でき，かつ具体的なライブラリ名を特定できたものを対象として，パッケージメタデータとの比較を行う．ライブラリ名を具体値まで特定できなかった呼び出しやパッケージへの内包を確認できなかった呼び出しは，依存関係との比較対象から除外する．

まず，抽出したライブラリ名から，そのライブラリを提

© 2026 Information Processing Society of Japan

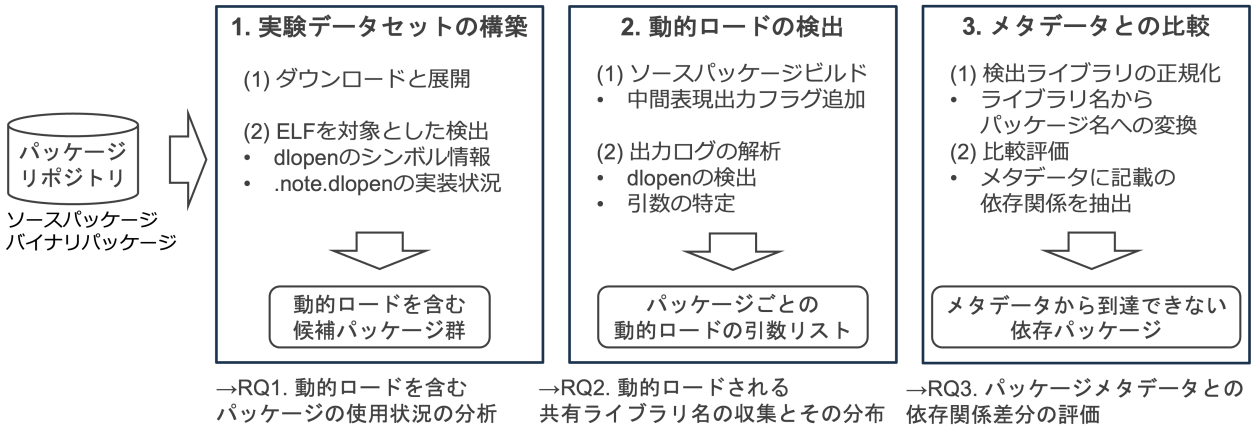


図 1: 実験パイプラインの概要

表 3: 動的ロード対象とパッケージ依存関係の分類

分類	判定内容
同一パッケージ内	呼び出し元パッケージ自身が対象ライブラリを提供する場合
直接の必須依存	Pre-Depends または Depends に直接含まれる場合
直接の任意依存	Recommends または Suggests に直接含まれる場合
推移的な必須依存	直接の依存関係には現れないが，Pre-Depends および Depends を推移的にたどることで到達できる場合
到達不能	上記のいずれにも該当しない場合

供するパッケージを特定する．ライブラリ名のみが指定されている場合には，ldconfig を用いて実際のライブラリパスを取得し，dpkg-query によってそのファイルを所有するパッケージを求める．加えて，apt-file を用いてリポジトリ全体のファイル索引からもライブラリを提供するパッケージを特定し，実験環境へのインストール状況によらず提供パッケージを取得できるようにする．また，再ビルドによって生成され，実験環境にインストールされていないパッケージについては，生成した deb ファイル一覧を用いて対応付ける．相対パスなどから提供パッケージを一意に特定できない場合は，未解決として区別する．最後に，呼び出し元パッケージの Pre-Depends，Depends，Recommends，Suggests および Provides を取得し，対象ライブラリを提供するパッケージとの関係を比較し，その結果を表 3 のように分類する．

5. 評価

5.1 実験対象とパイプライン到達数

実験対象は Debian 12 (bookworm) および Debian 13 (trixie) の main アーカイブに含まれる amd64 向けパッケージとした．各バイナリパッケージの deb ファイルを展開し，ELF 形式のファイルを抽出した．抽出した ELF について シンボルを解析し，dlopen を参照するファイルを検出した．さらに，各 ELF について.note.dlopen セクションの有無を確認した．この処理によって得られたバイナリパッケージを Source フィールドによってソースパッケージへ対応付け，以降のビルドおよび静的解析の候補集合とした．表 4 に実験対象と解析対象数を示す．Debian 12 では 34,169 個，Debian 13 では 37,475 個のソースパッケージのうち，事前のバイナリ検出によって dlopen の参照を確認したソースパッケージは，Debian 12 で 1,253 件，Debian 13 で 1,316 件であった．Debian 10 のパッケージで C 言語で開発されているパッケージが 6,000 件程度であり [14]，開発コードも半数程度である [15] ことを考慮すると，比較的多くのパッケージで dlopen が利用されているということが推測される．また，各 ELF における.note.dlopen の有無についても調査した結果，機能が実装されている Debian 13 では systemd と kmod，mimic の 3 件のパッケージで発見された．動的ロード自体は 1,000 件を超えるソースパッケージで確認されたことから，動的ロード対象を ELF へ明示的に記述する仕組みの利用は現時点では限定的である．

表 4: 対象パッケージと分析パッケージ数

項目	Debian 12	Debian 13
ソースパッケージ数	34,169	37,475
dlopen への参照を含む ELF が存在するパッケージ数	1,253	1,316
.note.dlopen を含むパッケージ数	0	3
ビルド成功数	944	938
dlopen を検出したパッケージ数	652	659
dlopen の呼び出し検出数	1,994	1,857

5.2 RQ1. 動的ロードはどの程度利用されているか？

バイナリ解析の結果，dlopen への参照を持つ ELF を含むソースパッケージは，Debian 12 で 1,253 件，Debian 13 で 1,316 件で，ソースパッケージ全体のそれぞれ約 3.7%，約 3.5%に相当する．これは，Nibbler[13] の報告とも同程

© 2026 Information Processing Society of Japan

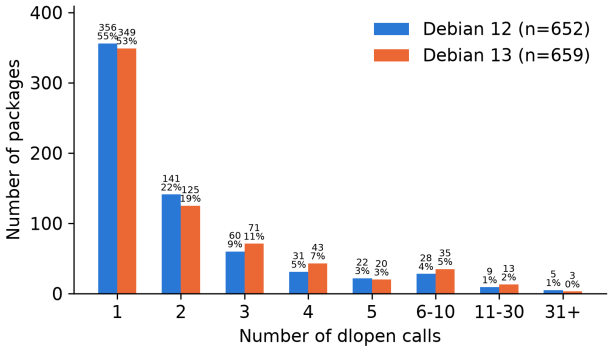


図 2: パッケージごとの動的ロード呼び出し数

度の規模である．また，それぞれのバージョンで，バイナリ解析によって抽出したソースパッケージのうち，944 パッケージ/938 パッケージについてビルドに成功し，中間表現の解析を行った．そのうち 652/659 パッケージから，1 つ以上の dlopen の呼び出しを確認した．これは，検証対象としたパッケージの約半数程度であり，確認された動的ロードの呼び出し箇所は合計 1,994 件/1,857 件であった．

パッケージごとの呼び出し箇所数を図 2 に示す．1 箇所のみで動的ロードを利用するパッケージが最も多く，呼び出しを確認したパッケージの 50%程度を占めた．一方で，一部のパッケージには多数の動的ロード処理が中間表現上に存在した．例えば，swt4-gtk では 198 箇所，wine では 36 箇所の呼び出しを確認した．これらには，実行時にドライバやモジュールなどを選択するソフトウェアが含まれていると考えられる．したがって，動的ロードは多数の Debian パッケージで利用されている一方，呼び出し箇所数はパッケージごとに大きく異なり，少数のパッケージに多数の呼び出しが集中する分布となっている．

5.3 RQ2: 静的解析による共有ライブラリ名特定可能性

中間表現から抽出した動的ロード呼び出しについて，引数からロード対象となる共有ライブラリ名を解析した．その結果を表 5 に示す．単一の具体的なライブラリ名を特定できた呼び出しは 660 箇所/499 箇所，複数の候補まで特定できたものは 120 箇所/122 箇所であった．したがって，ロード対象を具体的なライブラリ名として特定できた呼び出しは，780 箇所/621 箇所，全体の呼び出しのうち約 39.1%/約 33.4%となる．具体的には，vlc や jackd2 などのメディア系や，autofs や wine などの低レイヤコードを有するドライバ周辺のパッケージが存在していた．

一方で，引数の文字列の一部しか特定できない呼び出しや特定不能な呼び出しが，50%以上を占めており，静的解析では共有ライブラリ名を確定できないものも多数存在した．また，静的に特定できなかった呼び出しには，関数の引数や戻り値を介してライブラリ名が伝播するもの，メモリ上で文字列が構築されるもの，および設定や実行時入力によってロード対象が決定されるものが含まれていた．例えば，Apache の apr や nginx などパッケージでは引数を特定できておらず，ソースコードを確認してもどのようなライブラリが使用されるか推測しにくいものも多かった．このようなパッケージにおける使用ライブラリの特定手法や依存関係の追跡手法については今後の課題とする．

表 5: 動的ロード対象名の静的解析結果

解析分類結果	Debian 12		Debian 13	
	検出数	割合	検出数	割合
単一の文字列を特定	660	33.1%	499	26.9%
複数の文字列を特定	120	6.0%	122	6.6%
引数の一部を特定	173	8.7%	158	8.5%
特定不能	977	49.0%	1,003	54.0%
対象外	64	3.2%	75	4.0%
呼び出し検出数の合計	1,994	100.0%	1,857	100.0%

この結果から，動的ロードによって共有ライブラリを指定する呼び出しのうち，約 4 割については本手法によって具体的なライブラリ名を静的に特定できたものの，過半数の呼び出しについては，より高度な静的解析または実行時の観測が必要となることがわかった．

5.4 RQ3: メタデータとの比較による依存関係の差分

RQ2 で具体的なライブラリ名を一意に特定できた呼び出しについて，ライブラリを提供するパッケージの特定と依存関係の比較を行った．複数の候補が残るライブラリについては実行時にどの値が用いられるか確定できないため，対象外とした．その結果，引数から共有ライブラリ名を特定でき，かつバイナリパッケージとの整合が取れた，197 件/226 件の呼び出しを依存関係比較の対象とした．このうち，166 件/173 件については共有ライブラリ名から，提供するパッケージを一意に特定できた．一方，それ以外の呼び出しについては，提供するパッケージを特定できないケースや複数のパッケージが候補として残るケースがあった．そのため，以降では一意に対応付けることができた 166 件/173 件を対象として依存関係を評価する．

表 6 に，呼び出し元パッケージと，対象ライブラリを提供するパッケージとの関係を示す．最も多かったのは，対象ライブラリを提供するパッケージが Depends または Pre-Depends に直接記載されている場合であり，97 件/99 件であった．また，Recommends または Suggests に直接記載されているものが 13 件/27 件，必須依存を推移的にたどることで到達できるものが 6 件/4 件であった．一方，46 件/40 件については，既存のパッケージ依存関係から対象ライブラリを提供するパッケージへ到達できなかった．また，これらはそれぞれ 25 パッケージ/19 パッケージに分布していた．一例として，altos による libbluetooth3 が提供する libbluetooth.so.3 の動的ロード，htop による

© 2026 Information Processing Society of Japan

表 6: 動的ロード対象とパッケージ依存関係の対応

依存関係	Debian 12		Debian 13	
	件数	割合	件数	割合
同一パッケージ内	4	2.4%	3	1.7%
直接の必須依存	97	58.4%	99	57.2%
直接の任意依存	13	7.8%	27	15.6%
推移的な必須依存	6	3.6%	4	2.3%
依存関係から未発見	46	27.7%	40	23.1%
合計件数	166	100.0%	173	100.0%

libsensors5 が提供する libsensors.so.5 の動的ロードなどがメタデータに記載されない依存関係として確認された．ただし，これらには特定の機能を有効にした場合のみ利用されるライブラリや，任意のプラグインとして提供されるライブラリ，ビルドには含まれるものの実行には使用されないライブラリも含まれる可能性がある．そのため，依存関係が解決されない事例を直ちに依存記述の不備や実行に支障をきたす依存関係とはみなさず，これらを本実験パイプラインで観測された既存のパッケージメタデータのみからは把握できない動的ロード依存として捉える．

以上より，一意に提供パッケージを特定できた動的ロード対象のうち，Debian 12 と Debian 13 でそれぞれ約 7 割から 8 割はメタデータ依存関係を通じて動的ロードの依存関係を把握できた．一方，それ以外の約 2 割から 3 割で，動的ロードの呼び出しから検出できるパッケージの依存関係がパッケージメタデータ上の依存関係から到達できず，動的ロードでしか確認できない依存関係が一定数存在する可能性があることがわかった．

6. 議論

6.1 結果の考察

RQ1：動的ロードの利用規模．動的ロードは，一部の特殊なソフトウェアに限らず，バックエンドの機能やドライバなどを実行時に切り替えるための一般的な実装手段として利用されていた．一方で，利用状況には偏りがあり，多くのパッケージでは少数の呼び出しに留まるのに対し，一部のパッケージでは多数の動的ロードが集中する傾向も確認された．このことから，動的ロードは例外的な実装ではなく，機能を分離し，必要に応じて実行時に構成要素を選択するための仕組みとして多くのソフトウェアで利用されていると考えられる．また，Debian 12 と 13 の 2 つのバージョンで実験を行ったが，.note.dlopen の利用状況が異なるのみで，大きな差はなかった．

RQ2：ロード対象名の静的特定可能性．動的ロード対象の一部は静的に特定できるものの，関数の引数や戻り値，文字列操作などを介して決定されるものも多く，軽量な静的解析のみでは十分に把握できない．使用ライブラリを静的に特定できたパッケージにおいては.note.dlopen に記載することで依存関係の管理が自動化される見込みがある．一方で，本手法では特定できない場合でも，値の伝播や文字列構築を追跡するような静的解析の高度化によって追加的に特定できる余地はあるものの，設定ファイルや環境変数など実行時情報に依存するロード対象については，静的解析だけでは限界があり，実行時観測による補完が必要となる．

RQ3：パッケージメタデータとの対応．動的ロード依存の多くは既存のパッケージ依存関係から把握できる一方，メタデータから到達できないものも確認された．また，実際にはプログラムから直接ロードされるライブラリが，パッケージメタデータ上では推移依存としてのみ表現される場合もある．このことは，パッケージ間の依存関係と，実際のプログラムによるライブラリ利用関係が必ずしも一致しないことを示している．この差異は，依存グラフを用いて脆弱性の影響範囲を判定する場合に，実際の利用関係を正しく把握できない要因となり得る．ただし，メタデータから到達できない依存には，任意機能やプラグインとして意図的に依存関係へ含められていないものも存在する．そのため，これらを直ちに依存記述漏れとはみなさず，既存のパッケージメタデータのみでは把握できない動的ロード依存として解釈する必要がある．

6.2 知見とソフトウェア構成管理における課題

本研究の結果から，動的ロードは OS パッケージにおいて広く利用されている一方，その依存関係を既存の静的情報のみから把握することには限界があることがわかった．特に，動的ロード対象は必ずしも ELF の通常の依存情報に現れず，さらに dlopen の引数自体も静的に特定できない場合が多い．また，動的ロード対象を明示的に記録する .note.dlopen についても，現状の利用は限定的である．したがって，現在のパッケージメタデータや ELF 情報，開発者による明示的な宣言のみから，パッケージにおける動的ロード依存を網羅的に把握することは難しい．一方，本研究の静的解析によって一定の動的ロード対象を特定できたことから，メタデータの依存情報を静的解析によって補完することは有効である．しかし，設定ファイルや環境変数など実行時の状態によってロード対象が決定される場合には，静的解析のみでは実際のソフトウェア構成を確定できない．さらに本手法でライブラリ名を特定できなかったものも多く存在する上，静的に得られた潜在的な依存をすべて実際の構成要素として扱うと，利用されない任意機能まで含めてしまう可能性がある．

このことから，ソフトウェア構成管理では Runtime SBOM の重要性がより高まると考えられる．パッケージメタデータや静的解析によって，利用され得る構成を把握するとともに，実行時観測によって実際に利用された構成を確認することで，動的ロードを含むソフトウェア構成を

© 2026 Information Processing Society of Japan

より正確に表現できる．本研究では，依存関係の解決が実行時まで遅延される機構として dlopen を中心に扱ったが，同様の問題は環境による差異やプラグイン機構などにも共通しており，ソフトウェアの構成把握において実行時に決まる動的なソフトウェア構成の把握という共通課題が存在する．特に，脆弱性管理では，静的情報のみでは見落とされる依存（False Negative）と，静的には候補となるが実際には利用されない依存（False Positive）を区別する上で，Runtime SBOM がより重要となると考える．

6.3 本手法の限界

本研究の分析には，以下の制約がある．

動的ロードの検出範囲の制約：本研究では，ELF から dlopen への直接参照を確認できるパッケージを対象とした．そのため，関数ポインタや独自のラッパーなどを介したそのほかの動的ロードは検出できない可能性がある．また，中間表現上で検出しているため，実際のソースコード上の dlopen の呼び出し数とは一致しない．さらに，ビルドに失敗したパッケージについても個別に解析を進めることで追加の動的ロードを検出できる可能性があり，動的ロードの利用規模を過小評価している可能性がある．

静的解析による動的ロード対象の特定に関する制約：ユーザ設定や環境変数を読み込む場合などで左右されるロード対象は，本手法では完全に特定できない．より高度な静的解析によって追加的に取得できる場合もあるが，実行時情報に依存するものについては静的解析のみでの特定は困難である．

抽出したライブラリの実行時利用に関する制約：本研究で検出した動的ロードは，対象ビルドに含まれる潜在的な利用関係を表す．条件分岐や設定によっては実行されない場合がある．また，外部リポジトリ，ユーザが導入するプラグイン，ローカルに配置された共有ライブラリなどは分析対象外である．そのため，実際の利用状況を確認するためには実行時観測が必要となる．

7. おわりに

本研究では，Debian パッケージを対象として，dlopen による動的ロードの利用実態と，その共有ライブラリ依存が既存のパッケージメタデータからどの程度把握できるかを分析した．その結果，動的ロードは多数のパッケージで利用される一方，ロード対象を静的に特定できない呼び出しや，パッケージメタデータ上の依存関係から把握できない共有ライブラリが存在することを確認した．このことから，既存の静的な依存情報のみでは，動的ロードを含むソフトウェア構成を完全には把握できないことを示した．今後は，本研究で静的に抽出した動的ロード候補に対して実行時観測を行い，実際にロードされる共有ライブラリとの比較を行う．特に，パッケージのテストスイートや実際のワークロードを用いて動的ロードを観測することで，静的解析では特定できなかった依存の補完と，静的に抽出された候補が実際に利用されるかの確認を行う．これにより，静的解析による利用され得る構成と実行時観測による実際に利用される構成を組み合わせ，動的なソフトウェア構成管理と依存関係の特定を目指す．

参考文献

[1] Michael Kerrisk: dlopen(3) - Linux manual page, https://man7.org/linux/man-pages/man3/dlopen.3.html (参照 2026/08/20).

[2] LWN.net: Identifying Dependencies Used via dlopen(), https://lwn.net/Articles/969908/ (参照 2026/08/20).

[3] UAPI Group Specifications: UAPI.12 dlopen() Metadata for ELF Files | UAPI Group Specifications, https://uapi-group.org/specifications/specs/elf_dlopen_metadata/ (参照 2026/08/20).

[4] The Debian Project: Debian – Details of package dh-dlopenlibdeps in sid, https://packages.debian.org/sid/amd64/dh-dlopenlibdeps (参照 2026/08/20).

[5] Zhang, H. and Ali-Eldin, A.: RTrace: Towards Better Visibility of Shared Library Execution, *NDSS* (2026).

[6] systemd: systemd/package-notes: Tools to add packaging metadata to ELF files, https://github.com/systemd/package-notes (参照 2026/08/20).

[7] Xiao, Y., Kirat, D., Schales, D. L., Jang, J., Xing, L. and Liao, X.: JBomAudit: Assessing the Landscape, Compliance, and Security Implications of Java SBOMs, *NDSS* (2025).

[8] Qiu, T., Zhu, J., Chen, W. and Wei, J.: LiPSBOMaker: A Prototype of Multi-Stage Linux Distribution Package SBOM Generator, *Proceedings of the 34th ACM SIGSOFT International Symposium on Software Testing and Analysis*, pp. 56–60 (2025).

[9] Ali, H., Case, A. and Ahmed, I.: What You See Is Not What You Execute: Memory-Based Runtime SBOM Generation for Supply Chain Security, https://arxiv.org/abs/2606.22827 (2026). arXiv.

[10] Szumski, R.: Introducing the Real-time SBOM, https://edgebit.io/blog/realtime-sbom/index.html (参照 2026/08/20).

[11] Ben-Aharon, D.: Dynamic SBOM: Real-Time Monitoring for Enhanced Software Security, https://plaxidityx.com/ja/blog/engineering-ja/dynamic-sbom-monitoring-the-pulse-of-software-security (参照 2026/08/20).

[12] Prokhorenko, V., Islam, C. and Babar, M. A.: Analyzing the Evolution of Inter-package Dependencies in Operating Systems: A Case Study of Ubuntu, *Software Architecture, ECSA 2023*, Vol. 14212, pp. 233–249 (2023).

[13] Agadakos, I., Jin, D., Williams-King, D., Kemerlis, V. P. and Portokalidis, G.: Nibbler: debloating binary shared libraries, *Proceedings of the 35th Annual Computer Security Applications Conference*, pp. 70–83 (2019).

[14] Schwahn, O., Coppik, N., Winter, S. and Suri, N.: Assessing the state and improving the art of parallel testing for C, *Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis*, pp. 123–133 (2019).

[15] The Debsources developers: Statistics | Debian Sources, https://sources.debian.org/stats (参照 2026/08/20).

© 2026 Information Processing Society of Japan